

Complete Project Documentation

Production-grade Ansible automation for AWS EC2 discovery, Maven build, Docker image creation, and Docker Hub publishing.

Project name	Complete Maven Docker Deployment
Project root on server	/home/ubuntu/project
Control node prompt	(ansible-env) root@ip-172-31-37-193
Server OS	Ubuntu 26.04 LTS
AWS region	ap-south-1
Target group	env_dev EC2 instances tagged Environment=dev and Role=web
Generated on	July 5, 2026

Purpose of this document

This PDF explains the project in story format, documents every file, describes the server and AWS target details, and explains why the design has production-grade qualities.

Table of Contents

1. Project Story
2. Executive Summary
3. Production-Grade Qualities
4. Server and Environment Details
5. Architecture and Deployment Flow
6. Directory Structure
7. File-by-File Code Documentation
8. Runtime Execution Evidence
9. How to Run the Project
10. Security, Secrets, and Access Control
11. Troubleshooting Guide
12. Improvements for Enterprise Production
13. Conclusion

1. Project Story

This project starts with a common real-world deployment problem. A Java application needs to be built, packaged into a Docker image, and published so that it can be deployed consistently. Doing this manually on one machine is simple. Doing it repeatedly across a group of cloud servers is where mistakes begin: one server may miss Java, another may have Docker stopped, a repository may not be updated, or credentials may be handled insecurely.

The project solves that problem by turning deployment into code. The Ansible control node uses an AWS dynamic inventory to discover the correct EC2 instances automatically. Instead of maintaining a static IP list by hand, it asks AWS for running instances in the Mumbai region that carry the required tags. Once those hosts are found, the playbook connects to them as the ubuntu user, becomes root where needed, installs required software, prepares Docker, pulls the latest application code, builds the Maven project, builds a Docker image, logs in to Docker Hub using vault-protected credentials, and pushes the image.

In short, the project is a deployment pipeline written as Ansible infrastructure code. It combines configuration management, cloud inventory discovery, application build automation, containerization, secret management, and registry publishing into one repeatable workflow.

One-line project definition

An Ansible-based deployment automation project that discovers AWS EC2 web servers dynamically, prepares them, builds a Java Maven application, creates a Docker image, and pushes that image to Docker Hub.

2. Executive Summary

Area	Details
Automation tool	Ansible
Cloud provider	AWS EC2
Inventory type	Dynamic inventory using amazon.aws.aws_ec2
Target selection	Running EC2 instances with Environment=dev and Role=web tags
Application source	GitHub repository: https://github.com/jaiswaladi246/Boardgame.git
Build tool	Apache Maven
Runtime/package	OpenJDK 17 and Docker
Container registry	Docker Hub
Image name	satyanarayan4434/boardgame:latest
Secret handling	Ansible Vault for Docker Hub username and password/token
Execution result	6 hosts completed successfully with failed=0

The playbook is structured around a reusable Ansible role named `deploy_app`. The role contains the operational steps, while group variables store reusable configuration such as repository location and Docker image name. The main playbook, `site.yaml`, stays clean and focused: select hosts, load secrets, and run the role.

3. Why This Is a Production-Grade Project

This project has production-grade qualities because it is not a collection of manual commands. It is a repeatable, versionable, environment-aware automation workflow. The design shows several practices used in real DevOps

and platform engineering environments.

Production quality	How this project implements it
Infrastructure as code	Server preparation and build steps are written in Ansible YAML files, so the process can be reviewed, versioned, repeated, and improved.
Dynamic cloud inventory	The inventory is generated from AWS EC2 tags and state instead of a manually maintained host list.
Tag-based targeting	Only servers with Environment=dev and Role=web are selected, which prevents accidental execution on unrelated instances.
Reusable role structure	The deploy_app role separates deployment logic from the main playbook, making the project easier to extend.
Idempotent modules	apt, user, service, git, docker_image, and docker_login modules declare desired state instead of relying only on shell scripts.
Secret management	Docker credentials are loaded from vault.yaml, which should be encrypted with Ansible Vault.
Multi-host orchestration	The same deployment workflow ran across 6 EC2 instances in one command.
Observable execution	The play recap clearly shows ok, changed, skipped, unreachable, and failed counts for every host.
Task tags	Tasks are tagged, allowing focused operations such as only package installation, only Docker build, or only Docker push.

Honest production note

This is a strong production-grade foundation. For strict enterprise production, the hardening section later in this document should be applied: unique image tags, better failure handling, host key validation, CI integration, rollback strategy, and stronger IAM and secret policies.

4. Server and Environment Details

4.1 Control Node

Property	Value
Prompt	(ansible-env) root@ip-172-31-37-193:/home/ubuntu/project#
Private hostname/IP hint	ip-172-31-37-193
Project path	/home/ubuntu/project
Shell user shown	root
Ansible environment	ansible-env virtual environment
Operating system	Ubuntu 26.04 LTS
Main command used	ansible-playbook site.yaml -u ubuntu --ask-vault-pass

4.2 Managed EC2 Hosts

Host IP	Role in run	Result
13.201.26.240	Discovered AWS EC2 target	ok=9 changed=4 failed=0
13.232.11.19	Discovered AWS EC2 target	ok=9 changed=4 failed=0
13.235.8.252	Discovered AWS EC2 target	ok=9 changed=4 failed=0

Host IP	Role in run	Result
43.205.118.24	Discovered AWS EC2 target	ok=9 changed=4 failed=0
43.205.206.118	Discovered AWS EC2 target	ok=9 changed=4 failed=0
65.2.177.168	Discovered AWS EC2 target	ok=9 changed=4 failed=0

4.3 Required Access and Tools

- AWS credentials available to Ansible so the `amazon.aws.aws_ec2` inventory plugin can query EC2.
- SSH access from the control node to target EC2 instances as the `ubuntu` user.
- Privilege escalation allowed because the play uses `become: yes` for package, user, and service changes.
- Ansible collections: `amazon.aws` for dynamic inventory and `community.docker` for Docker modules.
- Docker Hub credentials stored in an encrypted `vault.yaml` file.

5. Architecture and Deployment Flow

5.1 High-Level Architecture

Layer	Component	Responsibility
Control layer	Ansible control node	Runs the playbook, reads configuration, queries AWS, and connects to target hosts.
Inventory layer	<code>amazon.aws.aws_ec2</code>	Discovers EC2 instances dynamically by region, state, and tags.
Target layer	EC2 web servers	Receive package installation, repository clone/update, Maven build, and Docker operations.
Source layer	GitHub	Stores the Boardgame application source code.
Build layer	Maven + Docker	Compiles the Java app and packages it as a Docker image.
Registry layer	Docker Hub	Receives the pushed image <code>satyanarayan4434/boardgame:latest</code> .

5.2 Deployment Flow

- 1 Ansible reads `ansible.cfg` and loads `inventory/aws_ec2.yaml` as the default inventory.
- 2 The AWS EC2 inventory plugin queries `ap-south-1` for running instances.
- 3 Inventory filters keep only instances tagged `Environment=dev` and `Role=web`.
- 4 Keyed groups create groups such as `env_dev` and `role_web` from EC2 tags.
- 5 `site.yaml` targets hosts: `env_dev` and enables `become: yes`.
- 6 `vault.yaml` is loaded to provide Docker Hub credentials securely.
- 7 The `deploy_app` role installs Maven, OpenJDK 17, Docker, and Git.
- 8 Docker is enabled and started on each host.
- 9 The Boardgame GitHub repository is cloned or updated under `/home/ubuntu/project/Boardgame`.
- 10 Maven builds the Java project with `mvn clean package`.
- 11 The Docker image is built from the application repository.
- 12 Ansible logs in to Docker Hub and pushes the image.

5.3 Text Architecture Diagram

```
Ansible Control Node
|
| reads ansible.cfg
| loads inventory/aws_ec2.yaml
v
AWS EC2 Dynamic Inventory
|
| region: ap-south-1
| filters: running + Environment=dev + Role=web
v
env_dev Target Hosts
|
| run deploy_app role with become: yes
v
Install Java/Maven/Docker/Git -> Clone GitHub repo -> Maven build
|
v
Build Docker image -> Docker Hub login -> Push image
```

6. Directory Structure

```
/home/ubuntu/project
|-- ansible.cfg
|-- group_vars
|   |-- all.yaml
|-- inventory
|   |-- aws_ec2.yaml
|-- roles
|   |-- deploy_app
|       |-- tasks
|       |-- main.yaml
|-- site.yaml
-- vault.yaml
```

Path	Purpose
ansible.cfg	Project-level Ansible configuration. It points Ansible to the AWS dynamic inventory and disables SSH host key prompts.
group_vars/all.yaml	Global variables shared by all target hosts, including repository path and Docker image settings.
inventory/aws_ec2.yaml	AWS dynamic inventory configuration that discovers EC2 instances by region, state, and tags.
roles/deploy_app/tasks/main.yaml	Main deployment task list for packages, Docker setup, Git clone, Maven build, image build, login, and push.
site.yaml	Main playbook entry point that targets env_dev hosts and applies the deploy_app role.
vault.yaml	Encrypted Ansible Vault file expected to hold Docker Hub credentials.

7. File-by-File Code Documentation

This section documents every project file. The code blocks match the project content provided, except vault.yaml, where secrets are intentionally represented safely.

ansible.cfg

```
[defaults]
inventory = ./inventory/aws_ec2.yaml
host_key_checking = False
[ssh_connection]
ssh_args = -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null
```

Setting	Explanation
inventory = ./inventory/aws_ec2.yaml	Makes the AWS dynamic inventory the default inventory for this project.

Setting	Explanation
host_key_checking = False	Avoids manual SSH host authenticity prompts. Useful for labs, but stricter production environments should enable host key validation.
ssh_args	Passes SSH options that disable strict host key checking and avoid writing known_hosts entries.

group_vars/all.yaml

```
git_repo_url: "https://github.com/jaiswaladi246/Boardgame.git"
repo_dir: "/home/ubuntu/project/Boardgame"
docker_image_name: "satyanarayan4434/boardgame"
docker_image_tag: "latest"
```

Variable	Explanation
git_repo_url	The source repository for the Boardgame application.
repo_dir	The path on each target host where the repository will be cloned or updated.
docker_image_name	The Docker Hub repository name used when building and pushing the image.
docker_image_tag	The image tag. Current value is latest.

inventory/aws_ec2.yaml

```
plugin: amazon.aws.aws_ec2
regions:
  - ap-south-1
filters:
  instance-state-name: running
  tag:Environment: dev
  tag:Role: web
hostnames:
  - ip-address
keyed_groups:
  - key: ec2_tags.Name
    prefix: name
  - key: ec2_tags.Environment
    prefix: env
  - key: ec2_tags.Role
    prefix: role
compose:
  ansible_host: public_ip_address
  ansible_user: "ubuntu"
```

Block	Explanation
plugin	Uses the amazon.aws.aws_ec2 dynamic inventory plugin.
regions	Limits discovery to ap-south-1.
filters	Finds only running EC2 instances with Environment=dev and Role=web tags.
hostnames	Uses instance IP address values as inventory hostnames.
keyed_groups	Creates Ansible groups from EC2 tags, including env_dev and role_web.
compose	Sets ansible_host to public_ip_address and ansible_user to ubuntu.

roles/deploy_app/tasks/main.yaml

```
---
- name: Install required packages
  apt:
    name:
      - maven
      - openjdk-17-jdk
      - docker.io
      - git
```

```

    state: present
    update_cache: yes
  tags:
    - packages
- name: Add ubuntu user to docker group
  user:
    name: ubuntu
    groups: docker
    append: yes
  tags:
    - docker_group
- name: Ensure Docker service is running
  service:
    name: docker
    state: started
    enabled: yes
  tags:
    - docker_service
- name: Clone or update Git repository
  git:
    repo: "{{ git_repo_url }}"
    dest: "{{ repo_dir }}"
    version: main
    update: yes
  tags:
    - git_clone
- name: Build Maven project
  shell: mvn clean package
  args:
    chdir: "{{ repo_dir }}"
  register: maven_build
  ignore_errors: yes
  tags:
    - maven_build
- name: Print Maven stdout if failed
  debug:
    var: maven_build.stdout
  when: maven_build.rc != 0
- name: Print Maven stderr if failed
  debug:
    var: maven_build.stderr
  when: maven_build.rc != 0
- name: Build Docker image
  community.docker.docker_image:
    name: "{{ docker_image_name }}"
    tag: "{{ docker_image_tag }}"
    source: build
    build:
      path: "{{ repo_dir }}"
  tags:
    - docker_build
- name: Log in to Docker Hub
  community.docker.docker_login:
    username: "{{ docker_username }}"
    password: "{{ docker_password }}"
  tags:
    - docker_login
- name: Push Docker image
  community.docker.docker_image:
    name: "{{ docker_image_name }}"
    tag: "{{ docker_image_tag }}"
    source: local
    push: yes
  tags:
    - docker_push

```

Task	Purpose	Production value
Install required packages	Installs Maven, OpenJDK 17, Docker, and Git.	Ensures every target has the same baseline tools.
Add ubuntu user to docker group	Allows ubuntu to run Docker commands.	Reduces repeated permission setup work.
Ensure Docker service is running	Starts and enables Docker.	Keeps container service available after reboot.
Clone or update Git repository	Gets application source from GitHub.	Keeps source deployment repeatable.

Task	Purpose	Production value
Build Maven project	Runs mvn clean package.	Builds the Java artifact before containerization.
Print Maven output if failed	Shows stdout and stderr when Maven fails.	Improves debugging.
Build Docker image	Builds the image from repo_dir.	Creates a deployable container image.
Log in to Docker Hub	Authenticates using vault variables.	Avoids hard-coded credentials in tasks.
Push Docker image	Pushes the local image to Docker Hub.	Publishes the artifact for downstream deployment.

site.yaml

```
---
- name: Complete Maven Docker Deployment
  hosts: env_dev
  become: yes
  vars_files:
    - vault.yaml
  roles:
    - deploy_app
```

Line/setting	Explanation
hosts: env_dev	Runs against hosts grouped from EC2 tag Environment=dev.
become: yes	Uses privilege escalation for package, service, user, and Docker operations.
vars_files: vault.yaml	Loads encrypted Docker Hub credentials.
roles: deploy_app	Executes the reusable deployment role.

vault.yaml

```
# This file is intentionally not printed in plain text.
# It should be encrypted using Ansible Vault and contain:
docker_username: "<docker-hub-username>"
docker_password: "<docker-hub-password-or-access-token>"
# Example commands:
# ansible-vault create vault.yaml
# ansible-vault edit vault.yaml
```

Important security note

The real vault.yaml should never be committed or shared in decrypted form. It should be encrypted with Ansible Vault and should contain Docker Hub credentials or, preferably, a Docker Hub access token.

8. Runtime Execution Evidence

The project was executed with the following command. The `--ask-vault-pass` flag tells Ansible to ask for the vault password at runtime so it can decrypt vault.yaml.

```
ansible-playbook site.yaml -u ubuntu --ask-vault-pass
```

8.1 Observed Task Progress

- Facts were gathered successfully from all 6 target hosts.
- Required packages were already present or installed successfully.
- The ubuntu user was confirmed as a member of the docker group.
- Docker service was running and enabled.

- The Git repository was cloned or updated on every host.
- Maven build completed successfully on every host.
- Docker image build completed on every host.
- Docker Hub login completed on every host.
- Docker image push completed on every host.

8.2 Play Recap

```
13.201.26.240 : ok=9 changed=4 unreachable=0 failed=0 skipped=2 rescued=0 ignored=0
13.232.11.19 : ok=9 changed=4 unreachable=0 failed=0 skipped=2 rescued=0 ignored=0
13.235.8.252 : ok=9 changed=4 unreachable=0 failed=0 skipped=2 rescued=0 ignored=0
43.205.118.24 : ok=9 changed=4 unreachable=0 failed=0 skipped=2 rescued=0 ignored=0
43.205.206.118 : ok=9 changed=4 unreachable=0 failed=0 skipped=2 rescued=0 ignored=0
65.2.177.168 : ok=9 changed=4 unreachable=0 failed=0 skipped=2 rescued=0 ignored=0
```

The important result is failed=0 and unreachable=0 for every host. This means the playbook successfully connected to all target machines and completed the deployment workflow.

8.3 Warning Explanation

Warning	Meaning	Recommended action
tags host variable deprecated	The AWS collection warns that the tags host variable will be removed in a future release.	Use ec2_tags instead of tags in inventory references and custom logic.
reserved name tags	Ansible treats tags as a reserved concept, so using a host variable named tags can create warnings.	Avoid creating or depending on a variable named tags.
Python interpreter discovery	Ansible discovered /usr/bin/python3.14 on each host.	Set ansible_python_interpreter explicitly if you want stable interpreter behavior.

9. How to Run the Project

9.1 Prerequisites

- Ansible installed inside the ansible-env environment.
- AWS credentials configured for the account and region containing the EC2 instances.
- amazon.aws and community.docker Ansible collections installed.
- Target EC2 instances running in ap-south-1 with tags Environment=dev and Role=web.
- SSH key access from the control node to the target instances as ubuntu.
- vault.yaml created and encrypted with Docker Hub credentials.

9.2 Useful Setup Commands

```
# Install collections if needed
ansible-galaxy collection install amazon.aws community.docker
# Create encrypted vault file
ansible-vault create vault.yaml
# Verify dynamic inventory
ansible-inventory -i inventory/aws_ec2.yaml --graph
# Run the deployment
ansible-playbook site.yaml -u ubuntu --ask-vault-pass
```

9.3 Running Individual Parts with Tags

```
# Only install required packages
ansible-playbook site.yaml --tags packages --ask-vault-pass
# Only clone or update repository
ansible-playbook site.yaml --tags git_clone --ask-vault-pass
# Only build Docker image
ansible-playbook site.yaml --tags docker_build --ask-vault-pass
# Only push Docker image
ansible-playbook site.yaml --tags docker_push --ask-vault-pass
```

Tags are valuable in production-style operations because they let engineers rerun a focused part of the workflow without repeating the entire playbook.

10. Security, Secrets, and Access Control

Security area	Current implementation	Recommendation
Docker credentials	Loaded from vault.yaml.	Keep vault.yaml encrypted and prefer Docker Hub access tokens instead of account passwords.
AWS access	Inventory requires AWS API permissions.	Use least-privilege IAM permissions limited to EC2 Describe operations for inventory discovery.
SSH trust	Host key checking is disabled.	For production, manage known_hosts and enable host key checking.
Privilege escalation	become: yes is used.	Limit sudo permissions to required commands where possible.
Image tagging	Uses latest.	Use immutable tags such as Git commit SHA, build number, or semantic version.
Secrets in output	Maven output is printed only on failure; Docker password is not printed.	Avoid debug output that could expose credentials.
Vault best practice Never paste decrypted vault contents into logs, documentation, or GitHub. Store only the variable names in documentation and keep the real values encrypted.		

11. Troubleshooting Guide

Problem	Likely cause	Fix
No hosts matched	EC2 tags, region, AWS credentials, or filters are wrong.	Run <code>ansible-inventory --graph</code> and verify tags <code>Environment=dev</code> and <code>Role=web</code> in <code>ap-south-1</code> .
SSH unreachable	Security group, key pair, username, or public IP issue.	Confirm port 22 is open from the control node and <code>ansible_user</code> is <code>ubuntu</code> .
Package install fails	APT lock, network issue, or repository problem.	Wait for apt locks, verify internet access, rerun packages tag.
Maven build fails	Application code, Java version, dependency, or test failure.	Check <code>maven_build.stdout</code> and <code>maven_build.stderr</code> printed by the playbook.
Docker build fails	Dockerfile issue or missing build artifact.	Verify the Maven target output and Dockerfile path inside <code>repo_dir</code> .
Docker login fails	Wrong Docker Hub credentials or token.	Edit encrypted vault.yaml and verify <code>docker_username</code> and <code>docker_password</code> .
Docker push denied	No permission to push to the repository.	Confirm the Docker Hub account owns or can write to <code>satyanarayan4434/boardgame</code> .

11.1 Useful Debug Commands

```
# Show dynamic inventory result
ansible-inventory -i inventory/aws_ec2.yaml --list
# Ping all env_dev hosts
ansible env_dev -m ping
# Run playbook with verbose logs
ansible-playbook site.yaml -u ubuntu --ask-vault-pass -vvv
# Check Docker service on target hosts
ansible env_dev -b -m service -a "name=docker state=started"
# Re-run only Maven build
ansible-playbook site.yaml --tags maven_build --ask-vault-pass
```

12. Improvements for Enterprise Production

The current project is a strong production-grade foundation. The following changes would make it stronger for an enterprise CI/CD environment.

Improvement	Why it matters
Use immutable Docker tags	Replace latest with commit SHA or build number so releases are traceable and rollback is possible.
Fail fast on Maven errors	Remove ignore_errors: yes or add an explicit fail task after printing logs so broken builds do not continue to Docker build and push.
Build once, deploy many	Building the same image on every EC2 host is expensive and can produce duplicate pushes. A dedicated build node or CI runner is cleaner.
Add container deployment	The current role builds and pushes an image. A complete deployment would also pull/run the image, expose ports, and add health checks.
Pin package and tool versions	Pinned versions improve reproducibility and prevent surprise changes from package updates.
Enable SSH host key checking	Production environments should verify host identity instead of disabling checks.
Set ansible_python_interpreter	Avoids future interpreter discovery drift on target hosts.
Use ansible-lint and Molecule	Automated testing catches role quality issues before production runs.
Move registry to AWS ECR if desired	ECR can integrate with AWS IAM and regional deployment workflows.
Add rollback strategy	Keep previous image tags and add tasks to redeploy a known-good image if the new one fails.

12.1 Suggested Hardening Patch Ideas

```
# Example group_vars improvement
docker_image_tag: "{{ lookup('env', 'BUILD_TAG') | default('latest', true) }}"
ansible_python_interpreter: /usr/bin/python3
# Example safer Maven pattern
- name: Build Maven project
  command: mvn clean package
  args:
    chdir: "{{ repo_dir }}"
  register: maven_build
  changed_when: true
- name: Fail if Maven build failed
  fail:
    msg: "Maven build failed. Check previous task output."
  when: maven_build.rc != 0
```

13. Conclusion

This Ansible project is a practical DevOps automation project because it connects several real production concerns: cloud discovery, repeatable server configuration, application build automation, container image creation, secure credential loading, and registry publishing. The successful play recap across 6 EC2 hosts proves that the workflow is operational, repeatable, and scalable beyond a single server.

Its strongest production-grade feature is the way it converts manual deployment steps into a clear, reviewable, reusable Ansible role. With the hardening improvements described in this document, it can evolve from a strong deployment automation foundation into a more complete enterprise CI/CD and runtime deployment system.